



Deprecate implicit conversions between `char8_t` and `char16_t` or `char32_t`

Document number:

[P3695R3](#)

Date:

2025-11-20

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Author:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3695/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/deprecate-unicode-conversion.cow



ABSTRACT: Implicit conversions between `char8_t` and `char16_t` or `char32_t` are bug-prone and thus harmful to the language. I propose to deprecate them.

§ Contents

- 1 **Revision history**
 - 1.1 Changes since R2
 - 1.2 Changes since R1
 - 1.3 Changes since R0
- 2 **Introduction**
 - 2.1 It's not hypothetical. This really happens.
 - 2.2 The underlying problem
- 3 **Scope**
 - 3.1 What about "safe" comparisons?



3.2	What about <code>char16_t</code> and <code>char32_t</code> ?
3.3	What about <code>char</code> ?
3.4	What about <code>wchar_t</code> ?
3.5	What about conversions with integers?
3.6	What comes after deprecation?
3.7	Why not make these conversions narrowing?
4	Impact on existing code
4.1	Replacement for deprecated behavior
5	Implementation experience
6	Wording
6.1	[conv.integral]
6.2	[expr.arith.conv]
6.3	[expr.static.cast]
6.4	[depr.conv.unicode]
7	Acknowledgements
8	References

§ 1. Revision history

§ 1.1. Changes since R2

R1 was seen by SG16 again, and the question of deprecating `char8_t` ↔ `wchar_t` conversions was reconsidered:



Poll 1: P3695R2: Recommend deprecating conversions between `char8_t` and `wchar_t`.

- Attendees: 10

•	SF	F	N	A	SA
	3	3	3	1	0

- Consensus.

Consequently, the following changes were made:

- reverted the deprecation of conversions with `wchar_t`
- improved the wording based on [isocpp-core] reflector feedback

§ 1.2. Changes since R1

R0 of the paper was seen by SG16, with the following poll results:



”

P3695R1: Recommend deprecating conversions between char and the charN_t types.

- Attendees: 10
- No objection to unanimous **dissent**.

P3695R1: Recommend deprecating conversions between char8_t and wchar_t.

- Attendees: 10
- No objection to unanimous consent.

P3695R1: Recommend deprecating conversions between char16_t and char32_t.

- Attendees: 10

•	SF	F	N	A	SA
	0	0	3	7	0

- Consensus against.

Consequently, the following changes were made:

- also deprecated conversion between `char8_t` and `wchar_t`
- changed title and abstract to reflect this new direction
- rewrote §3.2. [What about char16_t and char32_t?](#)
- expanded note on tautology warnings in §2.1. [It's not hypothetical. This really happens.](#)
- added §3.7. [Why not make these conversions narrowing?](#)
- restructured §6. [Wording](#) and added editorial notes

§ 1.3. Changes since R0

- limited deprecation to conversions involving `char8_t`; see §3.2. [What about char16_t and char32_t?](#)
- rebased §6. [Wording](#) on [N5014]

§ 2. Introduction

Implicit conversions between `char8_t` and `char32_t` invite bugs:



BUG: Until *very* recently, no major compiler would detect the following "bad comparison":



```
constexpr bool contains_oe(std::u8string_view str) {
    for (char8_t c : str)
        if (c == U'ö')
            return true;
    return false;
}
static_assert(contains_oe(u8"ö")); // fails?!
```

`c == U'ö'` always fails if `c` is a UTF-8 code unit because it is equivalent to `c == char32_t(0xf6)`, and a UTF-8 code unit cannot have this value.



BUG: An even more evil variation is a search which yields false positives:

```
constexpr bool contains_nbsp(std::u8string_view str) {
    for (char8_t c : str)
        if (c == U'\N{NO-BREAK SPACE}')
            return true;
    return false;
}
static_assert(contains_nbsp(u8"\N{CYRILLIC CAPITAL LETTER EL WITH MIDDLE
```

The assertion succeeds because Ъ (U+0520) is UTF-8 encoded as `0xd4, 0xa0`, and NBSP is U+00A0, so the `char32_t(0xa0)` value matches the second UTF-8 code unit of U+0520.



BUG: Such bad comparisons often don't occur directly, but within `<algorithm>`:

```
constexpr bool is_umlaut(char32_t c) {
    return c == U'ä' || c == U'ö' || c == U'ü';
}
// ...
constexpr std::u8string_view umlauts = u8"äöü";
static_assert(std::ranges::find_if(umlauts, is_umlaut) != umlauts.end())
```

Note that the "bad comparison" occurs between two `char32_t` in `is_umlaut`, which demonstrates that implicit conversions in general are bug-prone, not just comparisons. We obviously don't want to deprecate `char32_t == char32_t`.

Conversions "the other way" (e.g. `char32_t` → `char8_t`) are obviously bug-prone too because information is lost, but such bugs can already be caught by all major compilers' warnings, and they are problematic for the same reason as `int` → `short`, not because of anything specific to character types. The listed bugs are interesting *precisely because* no information is lost.

§ 2.1. It's not hypothetical. This really happens.

These kinds of bugs are not far-fetched hypotheticals either; I have written such bugs myself, and have had them contributed to my syntax highlighter [\[uilight\]](#), which makes extensive use of

`char8_t` and `char32_t`. Very early in development, I have realized how dangerous these implicit conversions are, so most functions in the style of `is_umlaut` have a deleted overload: 

```
constexpr bool is_umlaut(char8_t) = delete;
constexpr bool is_umlaut(char32_t c) {
    return c == U'ä' || c == U'ö' || c == U'ü';
}
```



NOTE: Compilers do have warnings which detect comparisons which are always `false`, but technically, `char8_t` can have the values `0xf6` and `0xa0`, so it is undetectable.

Using `char` may raise more tautology warnings because if `char` is signed, it can only hold values up to `127`, meaning it never compares equal to, e.g. `0xa0`.

§ 2.2. The underlying problem

The underlying problem is that `char8_t == char32_t` is `Car == Banana`. In general, it is meaningless to compare code units with different encodings.

To be fair, Unicode character types aren't strictly required to store Unicode code units. However, that is their primary purpose, and the assumption holds true for any Unicode *character-literal* and *string-literal*.

§ 3. Scope

I propose to deprecate implicit conversions between `char8_t` and `char16_t` or `char32_t`. As demonstrated above, these are extremely bug-prone. Conversions between `char16_t` and `char32_t` are not affected.

§ 3.1. What about "safe" comparisons?

In comparisons between code units, certain ranges of code points yield the expected result. For example, `u8'x' == U'x'` is `true` because all Unicode encodings are ASCII-compatible, so the numeric value of anything in the basic latin block ($\leq$ U+007F) will have the same single-code-unit value in UTF-8, UTF-16, and UTF-32.

However, even those should be deprecated because:

- Keeping these valid would essentially leak implementation details of UTF-8 into the set of implicit conversions in the C++ core language, which seems like unclean design.
- To rely on this "feature", the developer needs to memorize which code points are "safe to use". It is not obvious whether `c == U'€'` or `c == U'$'` are always safe (hint: the latter one is), and it's quite likely that someone uses this "feature" accidentally.
- It would make this "feature" (or lack thereof) harder to teach than it needs to be. The rule can be very simple: `char8_t` and some other character types cannot be converted to one

another. Simple rules are easy to teach.



§ 3.2. What about `char16_t` and `char32_t`?



NOTE: The following explanation assumes that `char16_t` and `char32_t` are used to store a UTF-16 and UTF-32 code unit, respectively.

Following some negative feedback on [\[ClangWarning\]](#), the proposal no longer seeks to deprecate conversions between `char16_t` and `char32_t`. While these conversions are not guaranteed to be meaningful, there are no false positives in comparisons of UTF-16 and UTF-32 code units, and the comparison is quite likely to be correct.



NOTE: There are no false positives in `char16_t` $\leftrightarrow$ `char32_t` because any code point in [U+0000, U+D7FF] or [U+E000, U+FFFF] is encoded using a single code unit equivalent to the code point value, in both UTF-16 and UTF-32.

Other code points are encoded using high surrogates ([0xD800, 0xDBFF]) and low surrogates ([0xDC00, 0xDFFF]). The corresponding surrogate code points in [U+D800, U+DFFF] cannot be encoded in UTF-32.

It is possible to have false negatives when searching for a UTF-32 code unit outside the Basic Multilingual Plane (BMP) in UTF-16 text. However, these searches are tautologically false because values $\geq 0x10000$ cannot (usually) be represented by `char16_t`, so compilers may catch some of them already.

It also also much less likely that `char16_t` $\leftrightarrow$ `char32_t` conversions actually manifest as a bug. An application that only uses, say, Basic Latin characters and German or Norwegian umlauts can use `char16_t` and `char32_t` interchangeably. By contrast, mixing `char8_t` with other Unicode character types will almost certainly blow up in the user's face if the application processes any kind of non-ASCII text.

Last but not least, UTF-8 is becoming the "default encoding", especially on the web, while UTF-16 is increasingly becoming a "legacy encoding". This makes it unattractive to raise warnings for `char16_t` when the surrounding code may exist mostly for compatibility purposes, and C++ users are not interested in sinking much time into its maintenance. Substantially more code may be affected by a `char16_t` $\leftrightarrow$ `char32_t` deprecation because both types were introduced in C++11, unlikely `char8_t`, which was added in C++20. See also [\[WikipediaEncodingPopularity\]](#):

”

Recently it has become clear that the overhead of translating from/to UTF-8 on input and output, and dealing with potential encoding errors in the input UTF-8, overwhelms any benefits UTF-16 could offer. So newer software systems are starting to use UTF-8. The default string primitive used in newer programming languages, such as Go, Julia, Rust and Swift 5, assume UTF-8 encoding. PyPy also uses UTF-8 for its strings, and Python is looking into storing all strings in UTF-8. Microsoft now recommends the use of UTF-8 for applications using the Windows API, while continuing to maintain a legacy "Unicode" (meaning UTF-16) interface.

In summary, in `char16_t` ↔ `char32_t` comparisons, there are no false positives, the only false negatives are tautologically false (warnings exist), bugs are unlikely to manifest because code points outside the BMP are relatively uncommon, and if deprecation warnings were raised, that may happen in low-priority legacy code.



NOTE: Not one participant in SG16 voted in favor of deprecating `char16_t` ↔ `char32_t` conversions; see §1.2. Changes since R1.

§ 3.3. What about `char`?

As recommended by SG16, I propose to leave `char` intact.

`char` ↔ `char8_t` in particular should not be deprecated because the encoding of both sides is likely UTF-8, in which case the conversion is obviously safe. Substantial amounts of code may already rely on this.

Furthermore, deprecating *any* conversion from `char` to other character types is a bad idea, and was unanimously recommended against by SG16. In some code bases, `char` is used *purely* for ASCII characters and strings. In such code bases, comparing `char` to any other character type is always correct, assuming that an ASCII-compatible encoding is used everywhere.

It may also be possible to deprecate conversions with `char` depending on ordinary literal encoding, but `char` is not necessarily using literal encoding, and doing so would invite non-portable code that fails to compile on e.g. EBCDIC platforms, to the great surprise of the author.

§ 3.4. What about `wchar_t`?

As recommended by SG16, `wchar_t` is also not affected by the deprecation.

While `wchar_t` almost certainly has a different encoding than `char8_t`, converting between it and `char8_t` is as problematic as the `char16_t` and `char32_t` conversions. However, while this is practically never the case, C2y permits UTF-8 as an encoding for `wchar_t`. Furthermore, `wchar_t` is not strictly guaranteed to be any wider than `char` or represent any more characters. Overall, it is a C compatibility hazard to touch `wchar_t`.

Besides `char8_t` ↔ `wchar_t`, `wchar_t` ↔ `char16_t` and `wchar_t` ↔ `char32_t` may be always correct, depending on the platform. Windows-only code can likely treat `wchar_t` and `char16_t` interchangeably, and Linux-only code may treat `wchar_t` and `char32_t` interchangeably.

§ 3.5. What about conversions with integers?

It is quite common to compare character types to integer types. For example, we may write `c <= 0x7f` to check whether a character falls into the basic latin block. There is nothing exceptionally bug-prone about comparing with say, `0x00A0` instead of `U'\u00A0'`, so we are not interested in deprecating character/integer conversions.

§ 3.6. What comes after deprecation?



The goal is to eventually remove these conversions entirely. Since the behavior is easily detected (§5. Implementation experience) and easily replaced (§4.1. Replacement for deprecated behavior), removal should be feasible within one or two revisions of the language.

Furthermore, I don't believe that having "tombstone behavior" would be necessary. That is, allowing the conversion to happen but making the program ill-formed if it happens. The reason is that `char8_t`, `char16_t`, and `char32_t` rarely appear in overload sets that include types that are not characters.



EXAMPLE: Without "tombstone behavior", the following code would eventually change its meaning:

```
void f(std::any);
void f(char32_t);

int main() {
    // Currently selects f(char32_t), would select f(std::any) in the fu
    f(u8'a');
}
```

§ 3.7. Why not make these conversions narrowing?

Another possible option (instead of deprecation or following deprecation) is to make the affected `char8_t` conversions narrowing conversions. This would make `char32_t{c}` for some `char8_t c` ill-formed, but the implicit conversion from `char8_t` to `char32_t` would remain valid.

There are multiple problems with this approach, which is why it is not proposed:

- `char8_t` to `char32_t` is a widening conversion, making the term "narrowing conversion" comically misleading.
- A long time has passed since C++11, and there is a lot of code using list-initialization now. This means that the "blast radius" of the change may still be quite large. If we accept that a non-trivial amount of warnings is raised in existing code, this half-measure seems unattractive.
- A lot of the problematic cases are not initialization, but comparisons as shown in §2. Introduction. Narrowing conversions play no role in equality comparison or in the usual arithmetic conversions.

§ 4. Impact on existing code

It is not trivial to estimate how much code would be affected by a deprecation like this. However, that is ultimately not what makes or breaks this proposal. The goal is not to deprecate a rarely used feature to give it new meaning, like `array[0,1]` prior to [P1161R3].

The goal is to deprecate a bug-prone and harmful feature to make the language safer.



The longer we wait, the more mistakes will be made using `char8_t` and other types. C++ will undoubtedly get improved support for the Unicode character types over time, making them used more frequently, so we better deal with this problem now than never.

§ 4.1. Replacement for deprecated behavior

If the new deprecation warnings spot a bug like in §2. Introduction, some work will be required to fix it, but the deprecation will have done its job.

If the comparison is obviously safe, such as `c == U'0'` with `char8_t c`, the resolution is usually trivial, like `c == u8'0'`. This could even be done automatically with tools like clang-tidy.

§ 5. Implementation experience

Corentin Jabot has recently implemented a `-Wcharacter-conversion` warning in Clang ([ClangWarning]), which is enabled by default. You can test this at [CompilerExplorer].

However the warning is more conservative than the proposed deprecation; it does not warn on "safe comparisons" (§3.1. What about "safe" comparisons?).

§ 6. Wording

The following changes are relative to [N5014].

§ [conv.integral]

Change [conv.integral] paragraph 1 as follows, and split it into two paragraphs:



1 A prvalue of an integer type can be converted to a prvalue of another integer type. The conversion is deprecated ([depr.conv.unicode]) if one of the types involved in the conversion is `char8_t`, and the other type is `char16_t` or `char32_t`.

2 A prvalue of an unscoped enumeration type can be converted to a prvalue of an integer type.

§ [expr.arith.conv]

Change [expr.arith.conv] paragraph 1 as follows:



Many binary operators that expect operands of arithmetic or enumeration type cause conversions and yield result types in a similar way. The purpose is to yield a common

type, which is also the type of the result. This pattern is called the *usual arithmetic conversions*, which are defined as follows:



- The lvalue-to-rvalue conversion ([[conv.lval](#)]) is applied to each operand and the resulting prvalues are used in place of the original operands for the remainder of this section.
- [...]
- Otherwise, each operand is converted to a common type C. The conversion is deprecated if one operand is of type `char8_t` and the other operand is of the types `char16_t` or `char32_t`. The integral promotion rules ([[conv.prom](#)]) are used to determine a type T1 and type T2 for each operand. Then the following rules are applied to determine C:
 - [...]



TIP: Integral promotion converts character types to `int` or `unsigned int`, so if we didn't add this wording, the conversion would not be deprecated.

§ [[expr.static.cast](#)]

Change [[expr.static.cast](#)] paragraph 5 as follows:



Otherwise, an expression *E* can be explicitly converted to a type *T* if there is an implicit conversion sequence ([[over.best.ics](#)]) containing no deprecated conversion from *E* to *T*, if [...]

[...] Otherwise, the result is direct-initialized from *E*.



EDITOR'S NOTE: There are no other deprecated conversions at the time of writing.

Do not change [[expr.static.cast](#)] paragraph 6:



Otherwise, the lvalue-to-rvalue ([[const.lval](#)]), array-to-pointer ([[conv.array](#)]), and function-to-pointer ([[conv.func](#)]) conversions are applied to the operand, and the conversions that can be performed using `static_cast` are listed below. No other conversion can be performed using `static_cast`.



EDITOR'S NOTE: The wording strategy is to get "nice and clean" prvalues out of the lvalue-to-rvalue conversion above, making the permitted conversion below easier to specify.

Immediately following [[expr.static.cast](#)] paragraph 6, insert a new paragraph:



An expression *E* of type `char8_t` can be explicitly converted to `char16_t` or `char32_t`, and vice versa. Such a conversion to a target type *T* is equivalent to `static_cast<T>`

```
(static_cast<unsigned char>(E)).
```

[Note: Integral conversions ([conv.integral]) between these types have the same effect and are deprecated, unlike this explicit conversion ([depr.conv.unicode]). — end note]



§ [depr.conv.unicode]

Insert a new subclause in [depr] between [depr.local] and [depr.capture.this], containing a single paragraph:



Unicode character conversions

[depr.conv.unicode]

The following conversions are deprecated:

- Integral conversions between `char8_t` and `char16_t` or `char32_t` ([conv.integral]).
- Arithmetic conversions from one operand of type `char8_t` and another operand of type `char16_t` or `char32_t` ([expr.arith.conv]).

[Example:

```
bool is_oe(char8_t c) {  
    return c == U'ö';  
}                                     // deprecated  
  
void f() {  
    char32_t c = u8'x';               // deprecated  
    char32_t c = u'x';               // OK, conversion from char16_t to char32_t  
    char8_t c = 'x';                 // OK, conversion from char to char8_t  
    is_oe(U'ö');                     // deprecated  
    is_oe(static_cast<char8_t>(U'ö')); // OK, explicit conversion ([expr.static.cast])  
    is_oe((char8_t)U'ö');             // OK, explicit conversion  
}
```

— end example]

§ 7. Acknowledgements

I thank Jens Maurer for reviewing the wording above and providing multiple suggestions for improvement.

§ 8. References

[N5014] *Thomas Köppe*. Working Draft, Programming Languages — C++ (2025-08-05)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf>



[μlight] *Jan Schultke*. `ascii_chars.hpp` utilities in μlight
<https://github.com/Eisenwave/ulight>

[ClangWarning] *Corentin Jabot*. [Clang] Add warnings when mixing different `charN_t` types
<https://github.com/llvm/llvm-project/pull/138708>

[CompilerExplorer] Demonstration of `-Wcharacter-conversion`
<https://compiler-explorer.com/z/8j9qqe8MY>

[WikipediaEncodingPopularity] Popularity of text encodings — Popularity internally in software
https://en.wikipedia.org/wiki/Popularity_of_text_encodings#Popularity_internally_in_software

[P1161R3] *Corentin Jabot*. Deprecate uses of the comma operator in subscripting expressions
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1161r3.html>